



Úvod do programování pro 1. ročník

Akademie programování pro střední školy

Ing. Pavel Čáp, DELTA – Střední škola informatiky a ekonomie, Pardubice

Obsah

Úvod a instalace	5
Výběr programovacího jazyka	7
Prohlídka programovacího prostředí	8
Soubor.....	9
Upravit	9
Zobrazit.....	10
Projekt.....	11
Sestavit.....	11
Ladit.....	12
Nástroje.....	12
Nápověda	13
První projekt.....	14
Základní datové typy	15
Celočíselné datové typy	16
Integer.....	16
Byte.....	17
Long.....	17
Reálné datové typy	18
Double	18
Float.....	18
Decimal.....	18
Matematické operátory.....	19
Třída Math.....	20
Odmocnina.....	20
Mocnina	20
Absolutní hodnota.....	20
Goniometrické funkce	20
Zaokrouhlování.....	21
Program „Výpočet slevy“.....	21
Program „Pythagorova věta“	22
Příklady na procvičení.....	23
Datový typ char	24

Datový typ string	25
Datový typ bool	26
Podmíněné příkazy	26
Program „Největší číslo ze tří“	27
Složené podmínky	28
AND	28
OR	28
Negace	28
Program „Výpočet přestupného roku“	29
Příklad na procvičení	29
Příkaz Switch	30
Program „Kolik má měsíc dnů“	31
Cykly	32
Cyklus For	33
Program „Součet sudých čísel“	33
Cyklus While	34
Cyklus Do-While	36
Hra „Hádání čísla“	37
Generování náhodného čísla	37
Příklady na procvičení	38
Jednorozměrné pole	39
Založení pole	39
Velikost pole	39
Čtení z pole	39
Zápis do pole	40
Seřídění pole	40
Otočení pole	40
Výpis pole	40
Cyklus Foreach	41
Program „Součet pole “	41
Program „Min a Max v poli “	42
Program „Naplnění pole náhodnými čísly“	43
Program „Překladač do Morseovy abecedy “	43

Dvourozměrné pole	45
Založení dvourozměrného pole	45
Zjištění velikosti 2D pole	46
Čtení z 2D pole	46
Zápis do 2D pole.....	46
Výpis 2D pole.....	47
Příklady na procvičení.....	47
Hra Lodě.....	48
Závěr.....	52

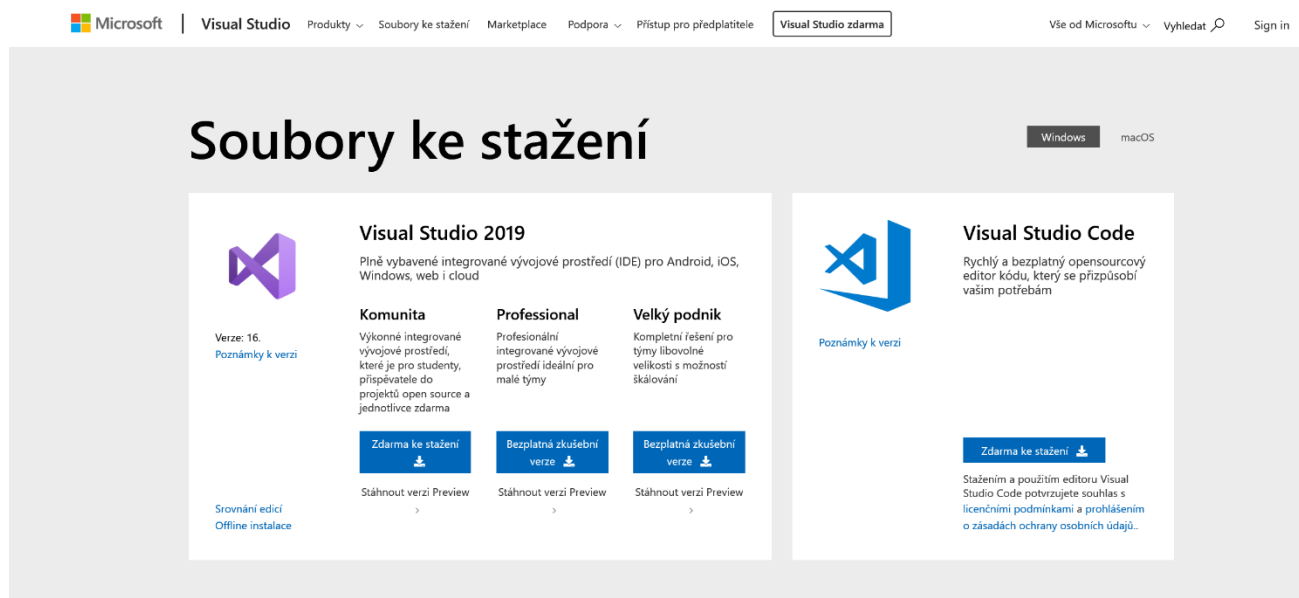
Úvod a instalace

Tato knížka je určená všem, co rádi poznávají nové věci. Knížka je psaná tak, aby i člověk, který nikdy neprogramoval, dokázal vytvořit zajímavé programy. Pokud máte rádi stavebnice a rádi poznáváte, co která součástka znamená a kam patří, tak věřte, že programování je jedna velká stavebnice s neuvěřitelným množstvím dílů. Postupně budete objevovat nové a nové věci a budete mít radost z každého nového programu. Vždy jsem se moc těšil na první rozbalení krabice, a tak to nebudeme zdržovat a pustíme se do toho.

Co budeme potřebovat pro naši práci? Nejdříve je nutné zjistit, jestli máte nainstalován operační systém Windows 10.

V menu Start klikněte na Nastavení – Systém – O systému

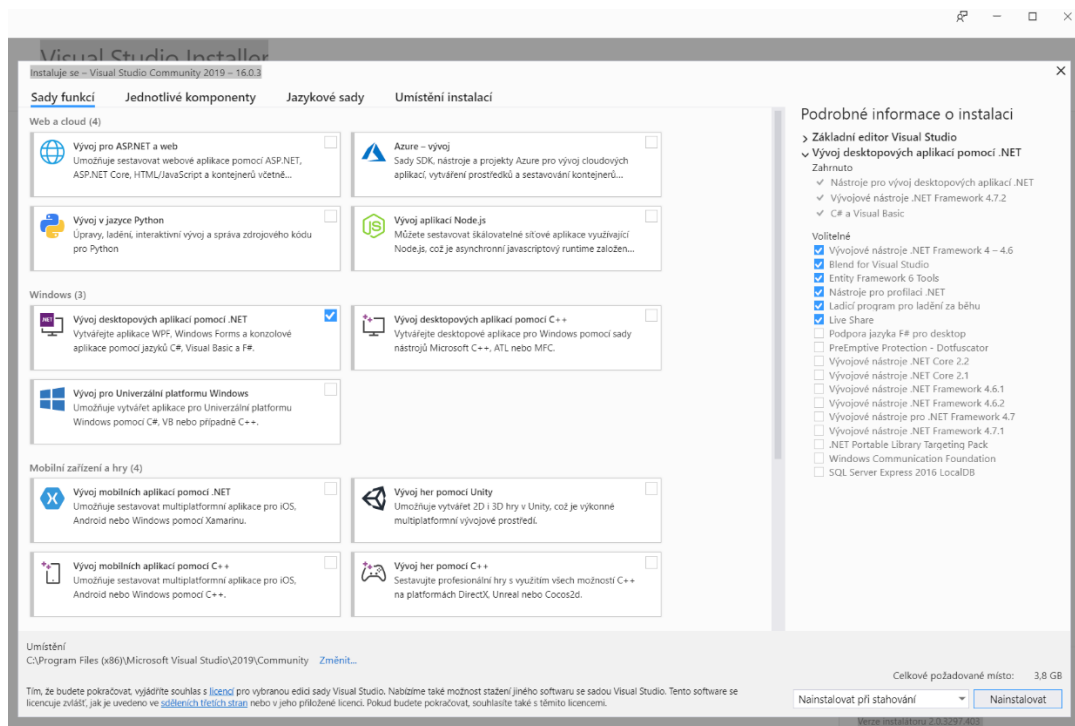
Další důležitou věcí je instalace Visual Studia 2019 od firmy Microsoft, které je skvělé a v Komunitě edici úplně zdarma. Najdete ho na stránkách Microsoftu, nebo ho můžete rovnou stáhnout na tomto odkazu: <https://visualstudio.microsoft.com/cs/downloads/>



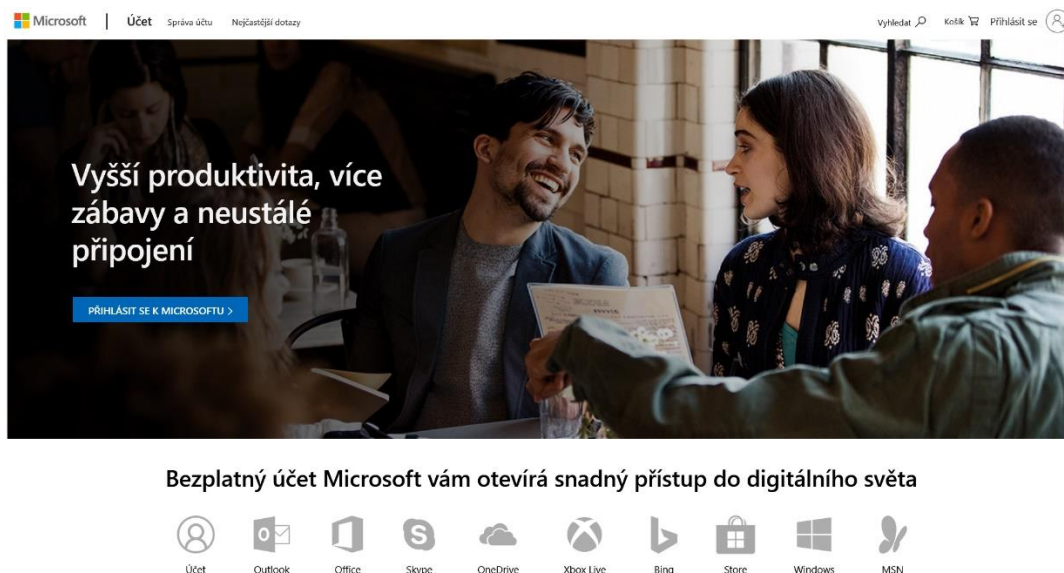
Webová stránka pro stažení Visual Studia 2019

Nejprve se stáhne a spustí instalační balíček. Postupně začne probíhat stahování jednotlivých částí Visual Studia. Průběžně budete informováni o stavu instalace. Následujícím kroku je třeba zaškrtnout Vývoj desktopových aplikací pomocí .NET a potvrdit tlačítkem Nainstalovat. Další možnosti v nabídce si můžete vždy později doinstalovat.

Průběh instalace



Pak už se zbývá jen přihlásit a pokud nemáte ještě vytvořený Live Id účet, pak si ho vytvořte zde <https://login.live.com> . Tento účet se vám bude hodit i pro jiné programy a služby od firmy Microsoft.

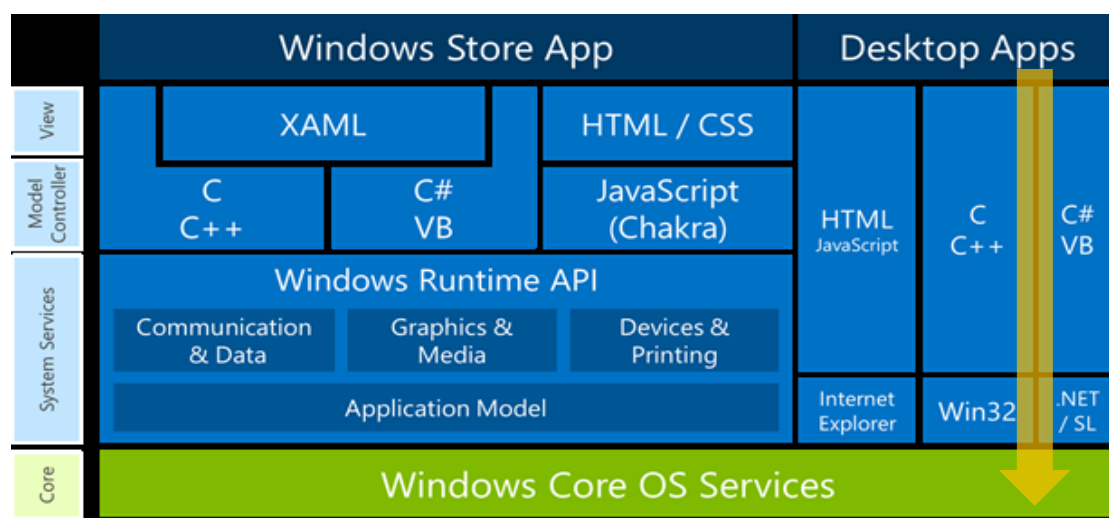


Registrace a vytvoření účtu

Doufám, že se vše podařilo, a v další kapitole se už pustíme do průzkumu programovacího prostředí.

Výběr programovacího jazyka

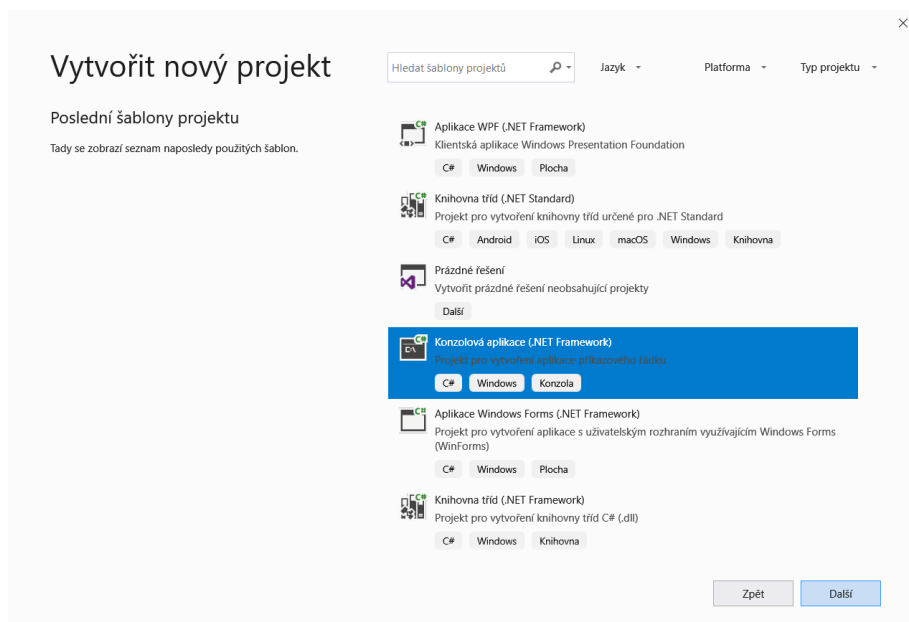
Programy, které budeme vytvářet, budou pracovat v novém běhovém prostředí. Toto běhové prostředí bylo vytvořeno pro aplikace s označením Desktopové. V základu si můžete vybrat jeden ze čtyř programovacích jazyků. My si pro naši práci vybereme programovací jazyk C#. C#, nebo také Csharp, vzniká již v roce 2002, vychází svojí syntaxí z jazyku C++ a Java. Jedná se moderní vysokoúrovňový objektový jazyk. Pro představu vnitřní struktury běhového prostředí, se podívejte na následující obrázek.



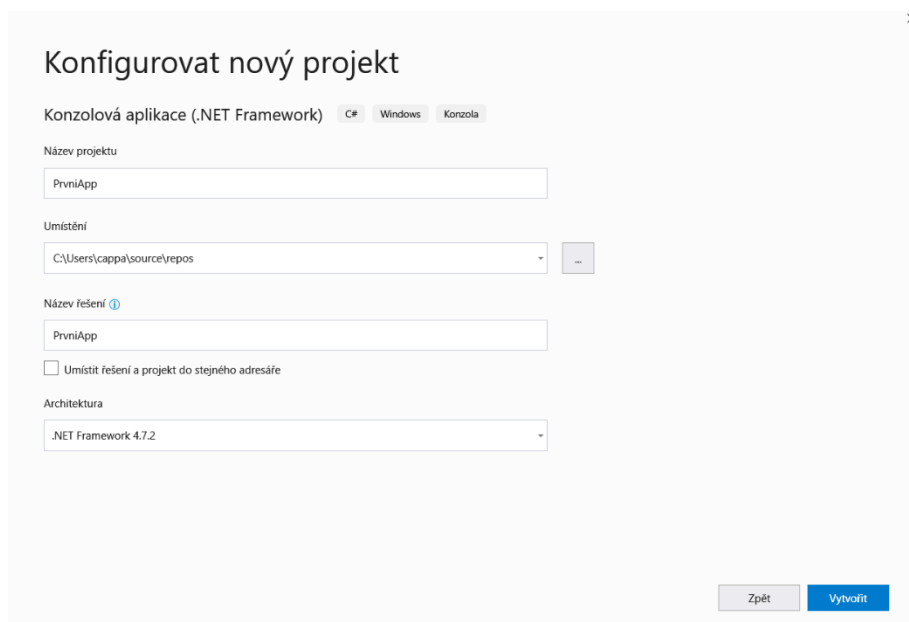
Šipka na obrázku ukazuje námi zvolenou variantu. Pěkně odshora.

Prohlídka programovacího prostředí

Nastal čas na vytvoření prvního projektu. Zapneme Visual Studio 2019. Zmáčkne tlačítko *Vytvořit nový projekt*.



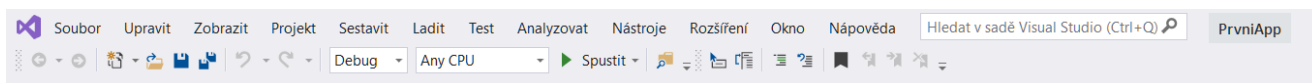
Vybereme *Konzolová aplikace* a potvrdíme tlačítkem *Další*.



V řádku *Název projektu* vyplníme název projektu *PrvniApp* a zkontrolujeme *Umístění*, kde bude projekt uložen. Nakonec potvrdíme *Vytvořit*.

Dále si popíšeme jednotlivé části rozložení obrazovky našeho projektu.

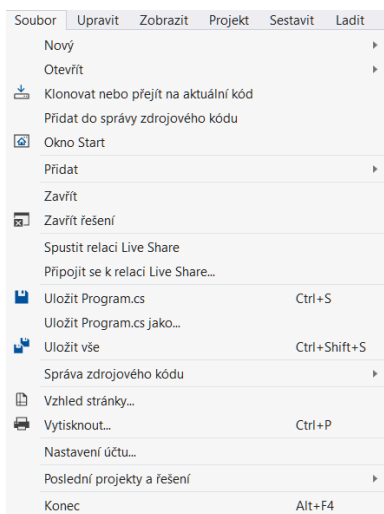
Menu – základní prostředek pro výběr jednotlivých nabídek. Nebudeme zde popisovat dopodrobna všechny nabídky, vybereme jen ty, které v této chvíli budeme potřebovat.



Základní lišta menu

Soubor

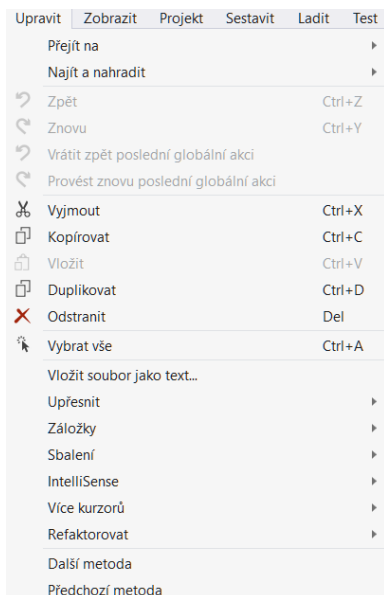
Menu → **Soubor** – slouží pro práci s projekty i jednotlivými soubory.



- *Nový* - Vytvoření nového projektu
- *Uložit vše* - Uložení všech souborů projektu
- *Otevřít* - Otevření projektu
- *Poslední projekty* - Otevření posledního projektu
- *Konec* - Ukončení Visual Studia

Upravit

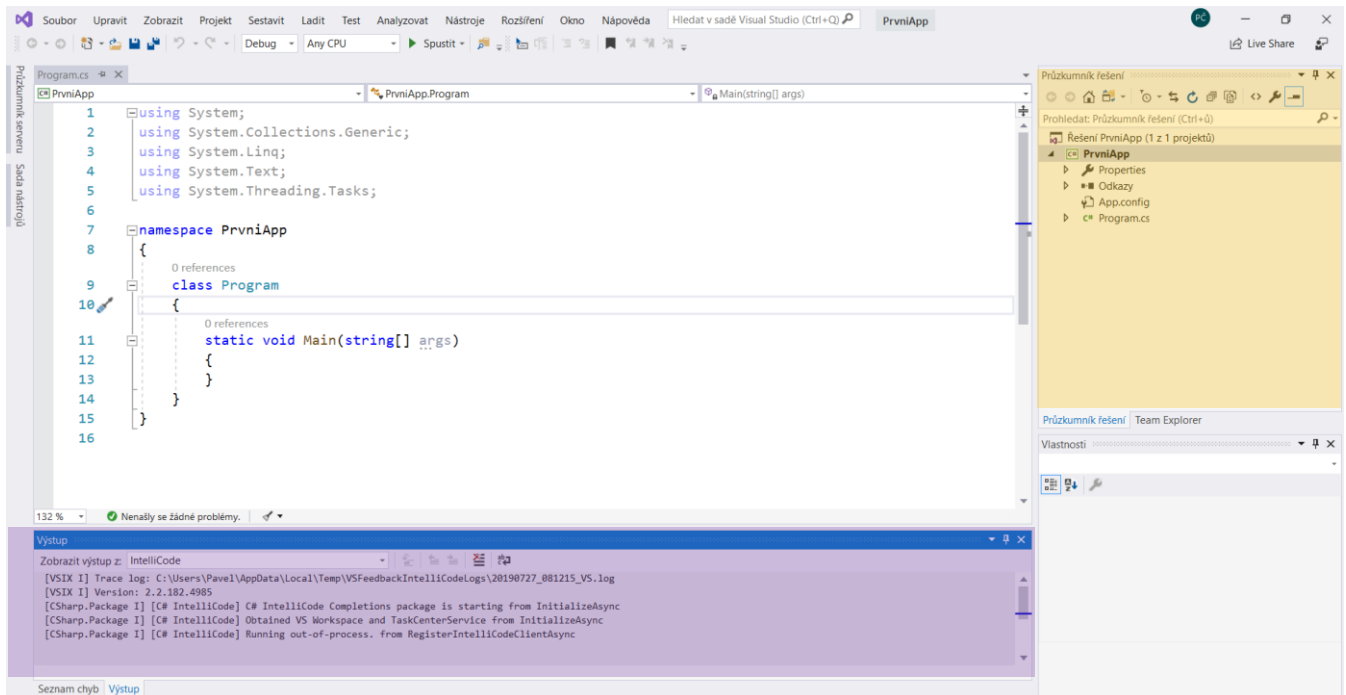
Menu → **Upravit** – slouží především pro editaci a vyhledávání v programu.



- *Vymout* - Vymout označenou část
- *Okopírovat* - Kopírovat označenou část
- *Vložit* - Vložit označenou část
- *Odstranit* - Smazat označenou část
- *Vybrat vše* - Označit vše
- *Najít a nahradit* - Vyhledat nebo nahradit text

Zobrazit

Menu → Zobrazit - slouží k zapnutí nebo vypnutí jednotlivých pracovních panelů na obrazovce, nebo přepínání jednotlivých typů souborů.



Zobrazit	Projekt	Sestavit	Ladit	Test
	Kód		F7	
	Otevřít			
	Otevřít v aplikaci...			
	Průzkumník řešení		Ctrl+Alt+L	
	Team Explorer		Ctrl+;, Ctrl+M	
	Průzkumník serverů		Ctrl+Alt+S	
	Okno záložek		Ctrl+K, Ctrl+W	
	Hierarchie volání		Ctrl+Alt+K	
	Zobrazení tříd		Ctrl+Shift+C	
	Okno definice kódu		Ctrl+;, D	
	Prohlížeč objektů		Ctrl+Alt+J	
	Seznam chyb		Ctrl+;, E	
	Výstup		Ctrl+Alt+O	
	Seznam úkolů		Ctrl+;, T	
	Panel nástrojů		Ctrl+Alt+X	
	Oznámení		Ctrl+;, Ctrl+N	
	Výsledky hledání			
	Ostatní okna			
	Panely nástrojů			
	Celá obrazovka		Shift+Alt+Enter	
	Všechna okna		Shift+Alt+M	
	Přejít zpět		Ctrl+-	
	Přejít vpřed		Ctrl+Shift+-	
	Další úkol			
	Předchozí úkol			
	Okno vlastností		F4	
	Stránky vlastností		Shift+F4	

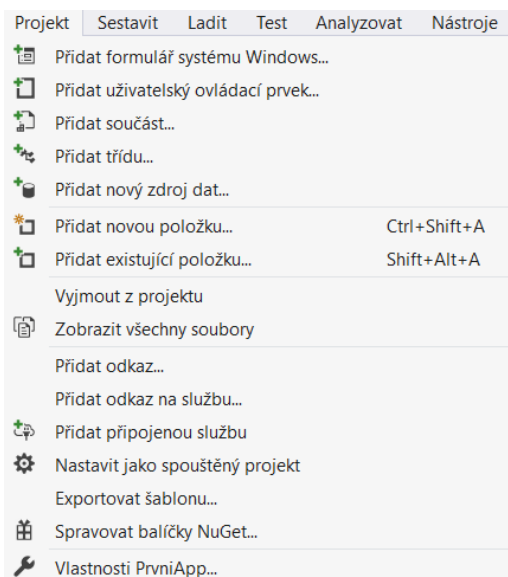
Průzkumník řešení - slouží k přehledu a správě souborů a složek v našem projektu. Přehledně zobrazuje všechny soubory včetně referencí a obrázků.

Výstup – výstupní panel, který informuje o spouštění, překladu, varováních a chybách aplikace. Je to užitečný pomocník při odlaďování programu.

Seznam chyb – ukáže aktuální seznam chyb s popisem.

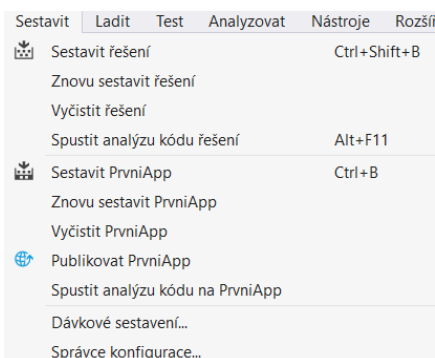
Projekt

Menu → Projekt – slouží k práci s projektem, přidávání a úpravě souborů projektu. Je možné také nastavovat a upravovat vlastnosti projektu. Všechny položky je možné nastavovat přes *Průzkumník řešení*.

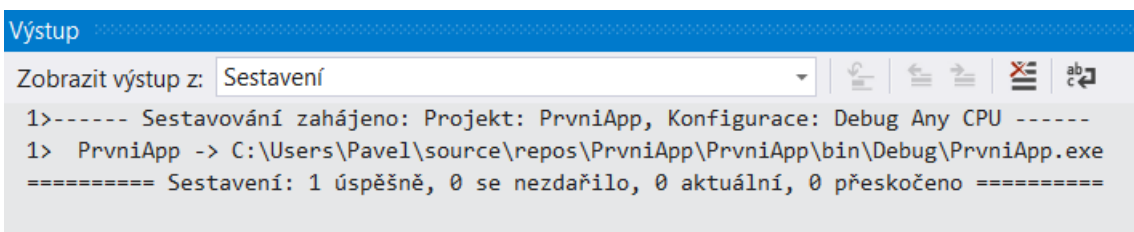


Sestavit

Menu → Sestavit – toto menu slouží k sestavení a kontrole vašeho programu. Při psaní kódu ve Visual Studiu dochází k automatickému vyhledávání syntaktických chyb (chyby ve špatném napsání příkazů) již během psaní. Výstup je automaticky směřován do panelu *Výstup*.

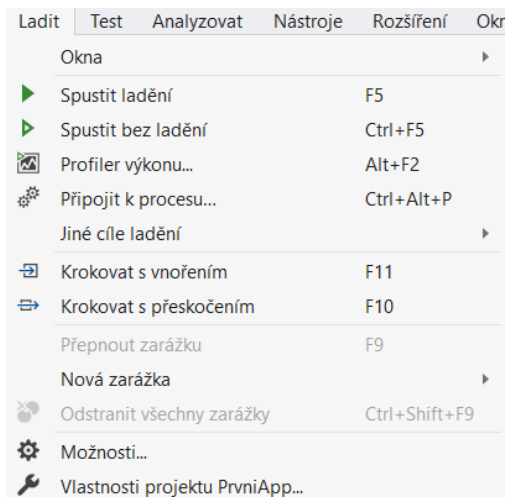


Výstup může vypadat třeba takto



Ladit

Menu → Ladit – toto menu slouží k puštění a odladění programu. Je zde možné nastavovat takzvané Zarážky (Breakpointy), které umožňují zastavení programu a zjištění například hodnot proměnných. Při spuštění programu dochází nejprve ke kontrole a sestavení a posléze i k vytvoření spustitelného souboru.



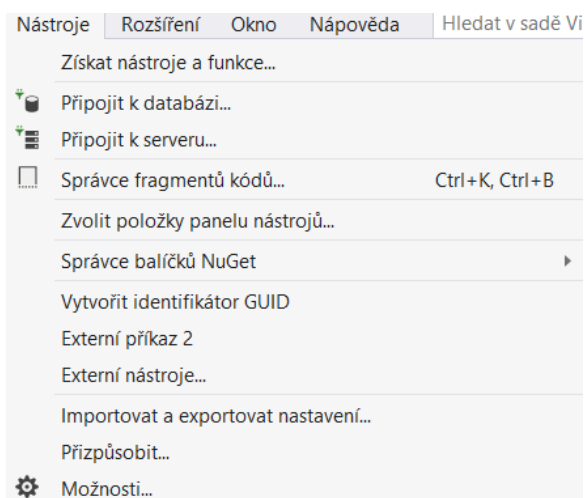
Ladit → Spustit ladění – spuštění s možností ladění programu.

Ladit → Spustit bez ladění – spuštění bez možností ladění programu.

Ladit → Krokovat s vnořením – možnost postupného krokování programu po jednotlivých příkazech.

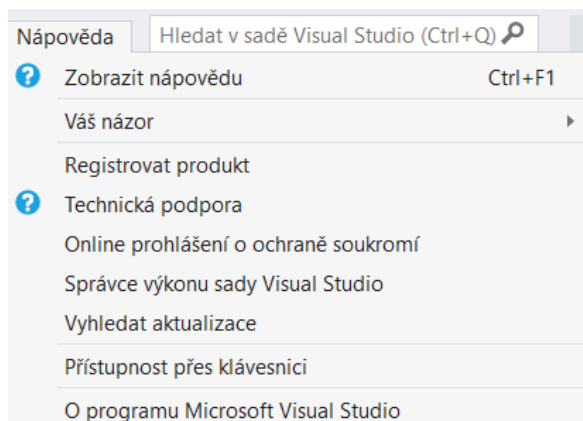
Nástroje

Menu → Nástroje → Možnosti – Velmi detailní nastavení programovacího prostředí. Tip – zkuste si pro začátek nastavit *Barevný motiv* v záložce *Prostředí* → *Obecné* z *Tmavého* na *Světlý* a naopak



Nápověda

Menu → *Nápověda* – Zde najdete nápovědu a pomoc při programování.



Pozorný čtenář si určitě všiml, že jsme přeskočili několik položek menu. Tato menu jsou stejně důležitá jako menu, která jsme prošli, ale pro vytvoření našeho prvního projektu je nebudeme potřebovat.

První projekt

Pokud jste si pečlivě prohlédli prostředí Visual Studia, tak se pustíme do dokončení naší aplikace. Většina knih o programování začíná aplikací „Ahoj Světe“ a my nebudeme výjimkou.

Nejprve si ukážeme, jak vytvořit vlastní komentáře v našem programu. Komentáře nijak neovlivňují chod programu, ale slouží pouze pro naší lepší orientaci v programu.

Komentář na jeden řádek vytvoříme za pomoci dvou lomítek.

```
// toto je náš komentář
```

Komentář do bloku vytvoříme za pomoci lomítka a hvězdičky.

```
/*  
    první řádek komentáře  
    druhý řádek komentáře  
    ....  
*/
```

Podívejte se na náš první program s komentáři. Do vašeho programu jsme přidali pouze dva příkazy (zvýrazněné části kódu). První vypíše do konzole řetězec "Ahoj světe" a druhý zastaví náš program do doby, než stisknu libovolnou klávesu. Program se bude postupně provádět řádek po řádku ve statické metodě Main. Program spustíte klávesou F5, nebo v menu *Ladit* → *Spustit ladění*.

```
using System; //připojení jmenného prostoru s názvem Systém  
  
namespace PrvniApp // jmenný prostor naší aplikace s názvem Prvniapp  
{  
    class Program // naše třída s názvem Program  
    {  
        static void Main(string[] args) // statická metoda s názvem Main  
        {  
            // vypíše Ahoj světe do konzole  
            Console.WriteLine("Ahoj světe");  
  
            // počká na stisk libovolné klávesy pro ukončení programu  
            Console.ReadKey();  
        }  
    }  
}
```

Pokud se vám vše povedlo, tak jste se právě stali programátory. Máte za sebou první opravdový program. Jistě ještě úplně všemu nerozumíte, ale toho se nebojte, v dalších kapitolách si o tom řekneme. Tak se nezdržujeme a pojďme rovnou na to.

Základní datové typy

Každý program, který napíšete, bude obsahovat nějakou proměnou. Proměnná není nic jiného než pojmenované místo v operační paměti počítače. Paměť počítače si můžete představit jako hodně velkou tabulku, kde v každé buňce může být uložena pouze 1 nebo 0 - třeba jako na v následující tabulce.

1	1	0	1	0	1	1	0	0	0	1	0	1	1	0	1	0	1	1	1	1	1	0	0	0	1	0	1	0	1
1	0	1	0	1	1	1	0	0	1	0	1	1	1	1	0	0	0	0	0	0	0	0	1	0	1	1	0	1	1
1	0	1	1	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Jedné buňce se říká bit, pokud spojíme dohromady 8 bitů, tak dostaneme jeden byte. Každá hodnota uložená v paměti musí být poskládaná jenom z jedniček a nul. Dobře, ale jak zapíšu třeba číslo 10? Je to docela jednoduché. Musíte si představit, že každá buňka má také přiřazenou nějakou hodnotu. Hodnota bitu je odvozena od binární soustavy $2^0=1$, $2^1=2$, $2^2=4$, $2^3=8$

Hodnota	128	64	32	16	8	4	2	1
Bit	0	0	0	0	1	0	1	0

Stačí sečíst hodnoty bitů, kde jsou jedničky, a dostaneme číslo v desítkové soustavě (soustava ve které počítáme) v našem případě $8+2=10$. Potom desítka je v paměti zapsána jako 1010.

Jak bude vypadat číslo 7?

Hodnota	128	64	32	16	8	4	2	1
Bit	0	0	0	0	0	1	1	1

Jak bude vypadat číslo 65?

Hodnota	128	64	32	16	8	4	2	1
Bit	0	1	0	0	0	0	0	1

Možná jste si všimli, že pokud číslo končí jedničkou, tak je liché. Naopak 0 je sudé. Jaké největší číslo se vejde do jednoho bytu (8 bitů)? Musíme dát všude jedničky, a pokud budete dobře počítat, vyjde vám 255.

Samozřejmě to jde spočítat i jednodušeji: 2^n-1 , kde n je počet bitů.

$$2^8-1=256-1=255$$

V případě čísel větších než 255 stačí přidat další bity a můžu uložit libovolná čísla. Potom třeba číslo 577 bude 1001000001

Hodnota	512	256	128	64	32	16	8	4	2	1
Bit	1	0	0	1	0	0	0	0	0	1

Každá proměnná v počítači je nějakého datového typu. Datové typy určují, co do proměnné mohu uložit, a na základě toho alokují (zaberou) potřebnou velikost paměti. Základní datové typy si teď ukážeme.

Celočíselné datové typy

Integer

Do celočíselného datového typu se ukládají celá čísla. Nejpoužívanější datový typ je Integer, jeho velikost je v základním tvaru 4 byty (32 bitů). Použití si ukážeme na jednoduchém programu. Použijeme naši aplikaci PrvniApp.

```
int A = 10;
```

tento příkaz založí celočíselnou datovou proměnnou typu Integer s názvem A a uloží do proměnné hodnotu 10. Znaménko = neznamená rovnost, ale zapiš hodnotu z pravé strany (10) do proměnné s názvem A. Do této proměnné můžete uložit čísla kladná i záporná do velikosti $2^{31}-1$ (-2 147 483 648 až 2 147 483 647). Jeden bit je použit pro znaménko, proto jenom 2^{31} .

Nyní upravíme projekt.

```
class Program // naše třída s názvem Program
{
    static void Main(string[] args) // statická metoda s názvem Main
    {
        // uloží do proměnné A hodnotu 10
        int A = 10;

        // vypíše proměnnou A
        Console.WriteLine(A);

        // počká na stisk libovolné klávesy pro ukončení programu
        Console.ReadKey();
    }
}
```

Program po spuštění vypíše hodnotu proměnné A.

Byte

Jedná se také o celočíselný datový typ. Do bytu lze uložit celá čísla od 0-255. Po úpravě bude náš program vypadat následovně.

```
class Program // naše třída s názvem Program
{
    static void Main(string[] args) // statická metoda s názvem Main
    {
        // uloží do proměnné A hodnotu 10
        byte A = 10;

        // vypíše proměnnou A
        Console.WriteLine("Hodnota v proměnné A je {0}",A);

        // počká na stisk libovolné klávesy pro ukončení programu
        Console.ReadKey();
    }
}
```

Jeho funkce bude úplně stejná, ale ušetříme místo v paměti. Všimněte si výpisu v metodě WriteLine, kde je použitý výpis do formátovaného řetězce znaků. Hodnota proměnné A se dosadí na místo {0}.

Long

Pokud budeme potřebovat uložit velmi velké celé číslo, můžeme použít datový typ Long. Jeho velikost je 8 bytu (64 bitů). U většiny programů si vystačíte s Integerem, ale přesto se Long někdy může hodit. Do této proměnné můžete uložit čísla kladná i záporná do velikosti $2^{63}-1$ (-9 223 372 036 854 775 808 až 9 223 372 036 854 775 807).

```
class Program // naše třída s názvem Program
{
    static void Main(string[] args) // statická metoda s názvem Main
    {
        // uloží do proměnné A hodnotu 10
        long A = 10;

        // vypíše proměnnou A
        Console.WriteLine("Hodnota v proměnné A je {0}",A);

        // počká na stisk libovolné klávesy pro ukončení programu
        Console.ReadKey();
    }
}
```

Reálné datové typy

Reálné datové typy slouží k ukládání čísel s desetinou čárkou. Většina programů, které mají za úkol počítat, používá tyto datové typy z důvodu přesnosti výpočtu.

Double

Jedná se o nejpoužívanější datový typ pro reálná čísla. Jeho velikost v paměti je 8 bytů (64 bitů). Přesnost double je 15-16 desetinných míst.

```
double prom = 10.5;
```

Pokud budete ukládat do proměnné konstantu, tak nezapomeňte, že desetinná čárka se zapisuje jako tečka. Tento řádek vytvoří proměnnou s názvem prom a uloží do ní hodnotu 10,5.

Float

Tento datový typ je o polovinu menší než double. Jeho velikost je 4 byte (32 bitů). Někdy se mu také říká singl. Přesnost je 7 desetinných míst.

Pozor na ukládání konstantních hodnot. Desetinné číslo se standardně přetypovává na double (double je dvakrát větší a proto ho nelze uložit do floatu), proto musíme ke konstantě doplnit f (float).

```
float prom = 10.5f;
```

Decimal

Poslední reálný datový typ je největší a nejpresnější pro ukládání desetinných čísel. Jeho přesnost je na 28-29 desetinných míst. Používá se zřídka, jen u výpočtů s nutností velmi přesných výsledných hodnot. U konstant nezapomeňte dopsat M.

```
decimal prom = 10.5M;
```

Matematické operátory

Právě jsme si prošli všechny číselné datové typy. Ještě je třeba si vysvětlit, jaké operátory lze u nich používat.

+ sčítání - sečte proměnné A a B a výsledek uloží do C

```
int A = 13;  
int B = 5;  
int C = A + B;
```

- odčítání - odečte proměnnou B od A a výsledek uloží do C

```
int A = 13;  
int B = 5;  
int C = A - B;
```

*** násobení** – vynásobí proměnnou A krát B a výsledek uloží do C

```
int A = 13;  
int B = 5;  
int C = A * B;
```

/ dělení – vydělí A děleno B a výsledek uloží do C. Pozor u celočíselných datových typů - ořezává desetinné místo, nejedná se o zaokrouhlování $13/5 = 2$!!!!

```
double A = 13;  
double B = 5;  
double C = A / B;
```

% modulo – modulo je celočíselný zbytek po dělení. Příklady:

$5\%3 = 2$ trojka se do pětky vejde jednou a zbydou 2

$7\%3 = 1$ trojka se vejde do sedmičky 2x a zbyde 1

```
int A = 13;  
int B = 5;  
int C = A % B;
```

Operátory mohou být i pro porovnání, ale ty si probereme později u podmíněných příkazů.

Třída Math

Než budeme pokračovat dále, bylo by dobré si ukázat jednu velmi zajímavou třídu, která se jmenuje **Math** a obsahuje spoustu užitečných metod pro počítání.

Odmocnina

Pokud budete například potřebovat druhou odmocninu, použijete metodu **Sqrt()**.

```
int A = 16;
double cislo = Math.Sqrt(A);
```

V proměnné **cislo** bude uložena hodnota 4.

Mocnina

Pro výpočet mocniny je připravená metoda **Pow()**.

```
int A = 10;
double cislo = Math.Pow(A,2);
```

První parametr je číslo, které chceme umocnit, a druhý parametr je na kolikátou. V našem případě 10^2 , což znamená, že v proměnné **cislo** bude uložena hodnota 100.

Absolutní hodnota

Pro výpočet absolutní hodnoty je připravená metoda **Abs()**.

```
int A = -10;
int cislo = Math.Abs(A);
```

Hodnota v proměnné **cislo** je 10.

Goniometrické funkce

Pro výpočet goniometrických funkcí naleznete metodu **Sin()**, **Cos()**, **Tan()** a další. Pozor, je třeba si uvědomit, že vstupní parametry se zadávají v radiánech. Pro přepočítání ze stupňů na radiány můžete zvolit následující vzorec:

```
radian = (stup * Math.PI)/180
```

V uvedeném příkladu je vypočítán $\sin 90^\circ$ a výsledná hodnota 1 uložena v proměnné **vysledek**.

```
double stup = 90;
double radian = (stup * Math.PI)/180;
double vysledek = Math.Sin(radian);
```


Zaokrouhlování

Pro zaokrouhlování čísel je možné použít metodu **Round(A,B)**. Vstupní parametr A je zaokrouhlované číslo a vstupní parametr B je na kolik desetinných míst.

```
double A = 9.458;  
double vysledek = Math.Round(A,2);
```

Potom ve výsledku bude uložena zaokrouhlená hodnota 9,46.

Třída Math obsahuje velké množství dalších metod, které si můžete projít a vyhledat. Úplný výpis metod s popisem naleznete na odkazu

<http://msdn.microsoft.com/cs-cz/library/system.math.aspx>

Program „Výpočet slevy“

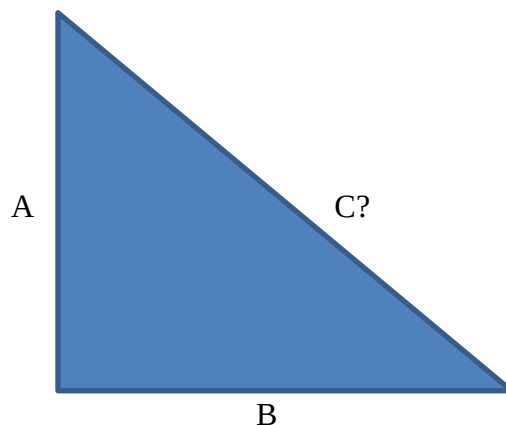
Pokud jste si pečlivě přečetli vše o reálných datových typech, tak nastal čas si je vyzkoušet na příkladu. Program, který vytvoříme, bude umět vypočítat slevy. Zadám současnou cenu a procenta slevy. Program zobrazí novou cenu zboží s uplatněnou slevou. Například, nové boty stojí 1000 Kč a prodavač mi dá slevu 15% a program spočítá, že nová cena je 850 Kč.

```
namespace Slevomatik  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            // Požádám uživatele o zadání původní ceny  
            Console.WriteLine("Zadejte cenu před slevou");  
            // založím proměnnou cena a přetypuji uživatelský vstup na double  
            double cena = Convert.ToDouble(Console.ReadLine());  
  
            // Požádám uživatele o zadání procentuální slevy  
            Console.WriteLine("Zadejte slevu v procentech");  
            // založím proměnnou procenta a přetypuji uživatelský vstup na double  
            double procenta = Convert.ToDouble(Console.ReadLine());  
  
            // založím proměnnou vysledek a provedu výpočet  
            double vysledek = cena / 100 * (100 - procenta);  
  
            // vypíšu proměnnou vysledek  
            Console.WriteLine("Nová cena je {0} Kč", vysledek);  
  
            // počkám na stisk libovolné klávesy pro ukončení programu  
            Console.ReadKey();  
        }  
    }  
}
```

Program „Pythagorova věta“

V této chvíli zkusíme vytvořit příklad, ve kterém použijeme některé z uvedených metod třídy Math. Náš program bude počítat podle Pythagorovy věty délku přepony v pravoúhlém trojúhelníku.

Máme pravoúhlý trojúhelník tvořený odvěsnami o délce A a B. Naším úkolem je zjistit délku přepony C. Budeme vycházet ze vzorce $C^2 = A^2 + B^2$.



1. Založte si nový projekt s názvem Pythagoras.
2. Opište následující program.

```
namespace Pythagoras
{
    class Program
    {
        static void Main(string[] args)
        {
            // Požádám uživatele o zadání první odvěsny
            Console.WriteLine("Zadejte velikost první odvěsny");
            // založím proměnnou odvesnaA a přetypuji uživatelský vstup na double
            double odvesnaA = Convert.ToDouble(Console.ReadLine());

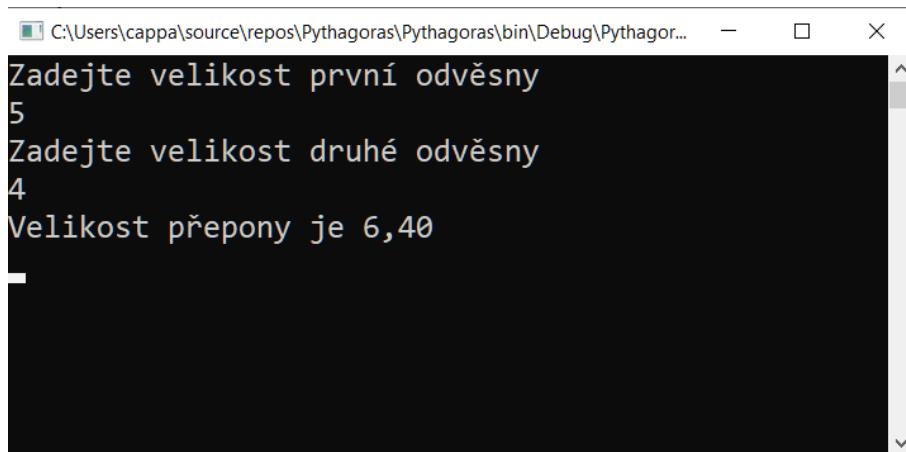
            // Požádám uživatele o zadání druhé odvěsny
            Console.WriteLine("Zadejte velikost druhé odvěsny");
            // založím proměnnou odvesnaB a přetypuji uživatelský vstup na double
            double odvesnaB = Convert.ToDouble(Console.ReadLine());

            // založím proměnnou preponaC a provedu výpočet
            double preponaC = Math.Sqrt(odvesnaA*odvesnaA + odvesnaB*odvesnaB);

            // vypíšu velikost přepony, 0:F2 určuje počet desetinných míst ve výpisu
            Console.WriteLine("Velikost přepony je {0:F2}", preponaC);

            // počkám na stisk libovolné klávesy pro ukončení programu
            Console.ReadKey();
        }
    }
}
```

3. Výsledek by měl po spuštění vypadat třeba takto:



```
C:\Users\cappa\source\repos\Pythagoras\Pythagoras\bin\Debug\Pythagor...
Zadejte velikost první odvěsny
5
Zadejte velikost druhé odvěsny
4
Velikost přepony je 6,40
```

Příklady na procvičení

1. Napište program pro výpočet objemu vody ve studni. Studna má tvar válce o poloměru R a výšce vody V . Zadávat R a V budeme v metrech a program vypíše objem v litrech.

Nápověda: Objem válce se vypočítá $V = 3.14 \times R^2 \times V$ a jeden m^3 vody má 1000 litrů.

2. Napište program pro výpočet celkového odporu dvou rezistorů zapojených v sérii a paralelně.

Nápověda: Výsledný odpor dvou rezistorů v sérii je $R = R_1 + R_2$ a výsledný odpor dvou paralelně zapojených rezistorů je $R = (R_1 \times R_2) / (R_1 + R_2)$

3. Napište program pro výpočet dojezdu automobilu. Do programu zadám spotřebu na 100 km a kolik mám v nádrži benzínu.
4. Napište program, do kterého zadáte čas. Zadám hodiny, minuty a sekundy. Program vypočítá kolik sekund uplynulo od začátku dne.

Datový typ char

Jedná se datový typ, do kterého lze uložit jeden znak.

```
char znak = 'A';
```

Po vykonání tohoto řádku bude v proměnné „znak“ uložen znak A.

V počítači mají všechny znaky svoji hodnotu i jako celé číslo. Například znak A má hodnotu 65, proto můžeme napsat toto:

```
int cislo_znaku = 'A';
```

Spojením znaků a čísel vznikne tabulka, které se říká ASCII (American Standard Code for Information Interchange). Její základní část je 7 bitová a obsahuje 128 znaků. Protože se do 128 znaků nevejdou národní sady, byla postupně rozšiřována na 8 a více bitů. My si ukážeme jen standardních 128 znaků. Vynecháme počáteční znaky 0-31, protože se jedná o takzvané neviditelné znaky.

číslo	znak	číslo	znak	číslo	Znak	číslo	znak
32	space	56	8	80	P	104	H
33	!	57	9	81	Q	105	I
34	"	58	:	82	R	106	J
35	#	59	;	83	S	107	K
36	\$	60	<	84	T	108	L
37	%	61	=	85	U	109	M
38	&	62	>	86	V	110	N
39	'	63	?	87	W	111	O
40	(	64	@	88	X	112	P
41	)	65	A	89	Y	113	Q
42	*	66	B	90	Z	114	R
43	+	67	C	91	[	115	S
44	,	68	D	92	\	116	T
45	-	69	E	93	]	117	U
46	.	70	F	94	^	118	V
47	/	71	G	95	_	119	W
48	0	72	H	96	`	120	X
49	1	73	I	97	A	121	Y
50	2	74	J	98	B	122	Z
51	3	75	K	99	C	123	{
52	4	76	L	100	D	124	
53	5	77	M	101	E	125	}

54	6	78	N	102	F	126	~
55	7	79	O	103	G	127	DEL

ASCII tabulka znaků

Datový typ string

Dalším datovým typem, s kterým se často budete setkávat ve svých programech, je typ string. Jedná se o datový typ pro ukládání řetězců znaků. Příkladem může být například proměnná jméno, do které uložíme slovo Petr.

```
string jmeno = "Petr";
```

Řetězec znaků může být libovolně dlouhý (omezuje ho pouze velikost paměti). S použitím řetězců jsme se již setkali v předchozích programech. Například při jakémkoli čtení uživatelského vstupu vrací metoda ReadLine() datový typ string a my ho pak převádíme na typ double.

Pokud bude potřeba zjistit počet znaků stringu, můžeme pro to použít vlastnost Length.

```
int pocet_znaku = jmeno.Length;
```

V našem případě bude v proměnné pocet_znaku hodnota 4, protože jméno Petr se skládá ze čtyř znaků.

Každý řetězec se skládá ze samostatných znaků, ke kterým se dá přistupovat pomocí indexu. Index je v podstatě ukazatel na jeden znak řetězce. Indexovat se začíná od nuly. Příklad indexů pro náš string můžete vidět v následující tabulce:

Index	0	1	2	3
Znak	P	e	t	r

Index se používá v hranatých závorkách za názvem proměnné. Na následujícím řádku je ukázka proměnné typu char ve které je uložen znak 'e' z našeho řetězce:

```
char znak = jmeno[1];
```

Datový typ bool

Do proměnných tohoto datového typu lze uložit pouze dvě hodnoty. První je hodnota **true (1)** a znamená pravda a druhá je hodnota **false (0)** a znamená lež. Tento datový typ se používá především pro podmíněné příkazy, o kterých se dozvíme v další kapitole.

```
bool rozhodnuti = true;
```

```
bool rozhodnuti = false;
```

Podmíněné příkazy

Podmíněné příkazy se používají všude tam, kde potřebujete rozdělit podle nějaké podmínky běh programu. Základní příkazem pro porovnání dvou a více proměnných je příkaz `if - else`.

Názorně si jeho funkci popíšeme na následujícím příkladu. Program řeší, jestli proměnná A je větší než B. Proměnná A a B může být libovolného hodnotového typu.

```
// pokud bude A > B, tak vykonaj kód ve složených závorkách pod if
if (A > B)
{
    // zde bude kód, který se vykoná, pokud bude platit podmínka A > B
}
else
{
    // zde bude kód, který se vykoná, pokud nebude platit podmínka A > B
}
```

Pokud bude za `if` nebo `else` následovat pouze jeden příkaz, není nutné psát složené závorky. Z vlastní zkušenosti doporučuji tyto závorky dodržovat, nic tím nepokazíte.

Samořejmě mezi proměnou A a B můžeme dávat i jiné operátory než větší. Jejich seznam zobrazuje následující tabulka.

operátor	popis
>	větší
<	menší
>=	větší nebo rovno
<=	menší nebo rovno
==	rovná se
!=	nerovná se

Program „Největší číslo ze tří“

Než budeme pokračovat, vyzkoušíme si napsat jednoduchý program, který nám najde největší číslo ze tří zadaných.

1. Založte si nový projekt s názvem MaxCislo.
2. Opište následující program.

```
namespace MaxCislo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Zadejte 1. číslo");
            // založím proměnnou A a přetypuji uživatelský vstup na integer
            int A = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("Zadejte 2. číslo");
            // založím proměnnou B a přetypuji uživatelský vstup na integer
            int B = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("Zadejte 3. číslo");
            // založím proměnnou C a přetypuji uživatelský vstup na integer
            int C = Convert.ToInt32(Console.ReadLine());

            int max = A;

            if(B > max)
            {
                max = B;
            }

            if (C > max)
            {
                max = C;
            }

            // vypíšu max
            Console.WriteLine("Největší číslo je {0}", max);

            // počkám na stisk libovolné klávesy pro ukončení programu
            Console.ReadKey();
        }
    }
}
```

3. Zkuste přepsat program tak, aby neobsahoval proměnnou max.

Složené podmínky

Příkaz `if` může mít i složitější konstrukci než porovnání mezi dvěma proměnnými. Může nastat případ, kdy budeme potřebovat vytvořit takzvanou složenou podmínku. Ukážeme si to na následujícím příkladu.

Obvykle jednou za čtyři roky nastává přestupný rok. Je to rok, kdy počet dnů v roce vzroste na 366. Měsíc únor má v přestupném roce 29 dní. Není tomu však vždy, a proto jsou stanoveny následující podmínky. Rok je přestupný tehdy, pokud jeho letopočet je beze zbytku dělitelný číslem 4 a zároveň není dělitelný 100, nebo je beze zbytku dělitelný číslem 400.

Zkusme vytvořit tuto podmínku pomocí složeného příkazu `if – else`.

Pokud budeme potřebovat, aby příkaz `if` obsahoval více jak jednu podmínku, musí existovat mezi jednotlivými podmínkami vztah. Tento vztah musí být jednoznačný a jeho výsledkem stejně jako u podmínek musí být pouze `True` a nebo `False`. Máme dva základní vztahy `AND`, `OR`.

AND – a zároveň (`&&`)

```
if(podminka1 && podminka2)
```

Pokud bude platit podmínka1 a zároveň podmínka2, tak celý `if` platí. Pokud jedna z podmínek platit nebude, tak celý `if` neplatí.

OR – nebo (`||`)

```
if(podminka1 || podminka2)
```

Pokud platí podmínka1, nebo podmínka2, tak celý `if` platí. Jen v případě, že neplatí žádná z podmínek, tak celý `if` neplatí.

Také je možné jednotlivé podmínky negovat

Negace – obrácená hodnota (`!`)

```
if(!podminka1)
```

Pokud podmínka1 bude platit, tak celý `if` neplatí, a naopak pokud podmínka jedna neplatí, tak celý `if` platí. Vykřičník otočí (neguje) platnost.

Ted' se vrátíme k našemu přestupnému roku. Nejprve je zde otázka, jak zjistíme, že rok je beze zbytku dělitelný? Stačí použít operátor modulo, a pokud se zbytek po dělení bude rovnat 0, tak je rok dělitelný beze zbytku.

Ted' už můžeme jít tvořit naši podmínku. Rok je přestupný tehdy, pokud jeho letopočet je beze zbytku dělitelný číslem 4 a zároveň není dělitelný 100.

```
if((rok%4 == 0)&&(rok%100 != 0))
```


Stačí už jen přidat druhou část. „nebo je beze zbytku dělitelný číslem 400“

```
if((rok%4 == 0)&&(rok%100 != 0)|| (rok%400 == 0))
```

Hotovo, pokud tato podmínka bude platit, je rok přestupný. Samozřejmě už existuje ve třídě `DateTime` metoda `IsLeapYear`, která zjišťuje to samé, ale to bychom si tak pěkně neprocvičili složené podmínky.

```
if(DateTime.IsLeapYear(rok))
```

A teď napíšeme program pro zjišťování přestupného roku.

Program „Výpočet přestupného roku“

1. Založte si nový projekt s názvem `PrestupnyRok`.
2. Opište následující program

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Zadejte letopočet");
        // založím proměnnou rok a přetypuji uživatelský vstup na int
        int rok = Convert.ToInt32(Console.ReadLine());

        // vytvořím podmínku pro přestupný rok
        if ((rok % 4 == 0) && (rok % 100 != 0) || (rok % 400 == 0))
        {
            Console.WriteLine("Rok je přestupný");
        }
        else
        {
            Console.WriteLine("Rok je nepřestupný");
        }

        // počkám na stisk libovolné klávesy pro ukončení programu
        Console.ReadKey();
    }
}
```

Příklad na procvičení

Vytvořte program, který zjistí, jestli je číslo sudé nebo liché.

Nápověda: Číslo je sudé tehdy, pokud jeho celočíselný zbytek po dělení dvěma nula.

Příkaz Switch

V případě, že budete potřebovat rozdělit běh programu na více jak dvě části, je vhodné použít příkaz Switch. Switch je anglicky přepínač a stejně tak se chová i v programu. Základní konstrukce vypadá následovně.

```
// do proměnné A uložím nějaké číslo, podle kterého budu přepínat
int A = ?;

// výstupní řetězec
string retez;

// vstupní proměnná A, podle které se bude přepínat
switch (A)
{
    // za příkazem case následuje číslo, se kterým se porovnává
    // hodnota v proměnné A
    // pokud se hodnoty rovnají, vykonají se příkazy za dvojtečkou
    // pokud program dojde k příkazu break, ukončí se celý switch
    // pokud není žádný příkaz za dvojtečkou, tak se program propadne
    // za následující case a pokračuje, dokud nenarazí na break

    case 5:
    case 10: retez = "A se rovná 10 nebo 5"; break;
    case 20: retez = "A se rovná 20"; break;
    case 30: retez = "A se rovná 30"; break;
    case 40: retez = "A se rovná 40"; break;

    // pokud se nerovná hodnota proměnné žádné hodnotě za case
    // vykoná se následující řádek
    default: retez = "A se nerovná se žádnému číslu"; break;
}
```

Je potřeba si uvědomit, že pokud budeme potřebovat, aby se jednotlivé větve programu propadaly, tak nesmí za dvojtečkou následovat žádný příkaz.

V následujícím programu si prakticky ukážeme použití Switche. Program bude po zadání čísla měsíce vypisovat, kolik má daný měsíc dnů. Například zadám 3 (březen) a program vypíše, že má měsíc 31 dní.

Tabulka měsíců a počet jejich dnů.

Číslo měsíce	1	2	3	4	5	6	7	8	9	10	11	12
Počet dnů	31	28-29	31	30	31	30	31	31	30	31	30	31

Program „Kolik má měsíc dnů“

1. Založte si nový projekt s názvem PocetDnu.
2. Předtím, než napíšeme program, je třeba si uvědomit, jak co nejlépe vytvořit strukturu Switche. Můžeme napsat dvanáct příkazů Case s hodnotami 1-12, ale existuje daleko jednodušší způsob. Pokud se pozorně podíváte na tabulku měsíců, tak zjistíte, že měsíce, které mají 31 dní, jsou nejčastější (7x). Následují měsíce, co mají 30 dní (4x) a nakonec únor, který má 28-29 dní podle přestupného roku. V našem programu, vzhledem k tomu, že nezačínáme letopočet, budeme počítat s 28 dny. Využijeme propadávání, což nám celý program zjednoduší.
3. Další možností je využít default pro měsíce, co mají 31 dní, ale potom se vyplatí před příkazem Switch udělat kontrolu čísla měsíce, jinak by program po zadání jiného čísla než 1-12 psal, že tento měsíc má 31 dní.
4. Napíšeme následující program.

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Zadejte číslo měsíce");
        // založím proměnnou mesic a přetypuji uživatelský vstup na int
        int mesic = Convert.ToInt32(Console.ReadLine());

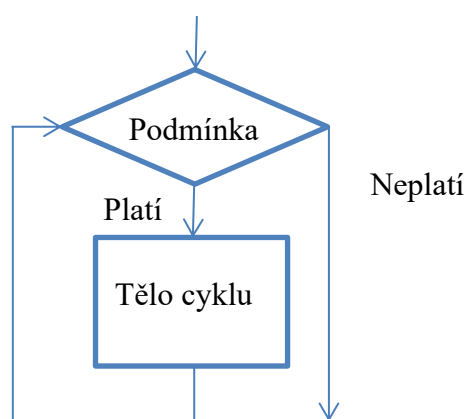
        // kontroluji rozsah proměnné M, jestli je mezi 1-12
        if (mesic >= 1 && mesic <= 12)
        {
            // vstupní proměnná mesic, podle které se bude přepínat
            switch (mesic)
            {
                // pro 4,6,9,11 počet dnů je 30
                case 4:
                case 6:
                case 9:
                case 11: Console.WriteLine("Měsíc má 30 dnů"); break;
                // pro únor 28
                case 2: Console.WriteLine("Měsíc má 28 dnů"); break;
                // ostatní měsíce mají 31 dní
                default: Console.WriteLine("Měsíc má 31 dnů"); break;
            }
        }
        // v případě špatného rozsahu vypiš
        else
        {
            Console.WriteLine("Špatné zadání, zadej číslo měsíce od 1-12");
        }

        // počkám na stisk libovolné klávesy pro ukončení programu
        Console.ReadKey();
    }
}
```

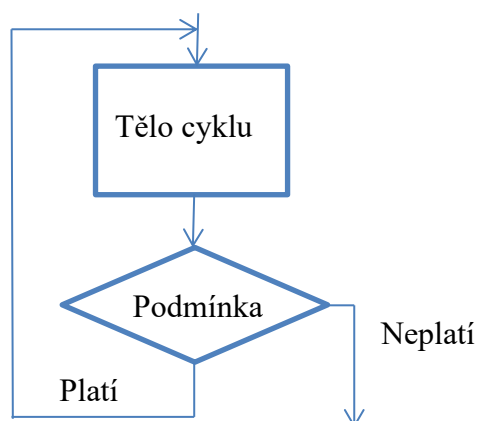
Cykly

Pokud to s programováním myslíte opravdu vážně, tak se bez cyklů neobejdete. Cyklus umožní část kódu několikrát opakovat. Část kódu, která se opakuje, se nazývá tělo cyklu. Cyklů je celá řada, ale v základě je můžeme rozdělit následovně.

1. Cykly s podmínkou na začátku. Jedná se o velmi užívaný typ cyklu, kde nejprve dochází ke kontrole podmínky a v případě, že podmínka platí, dojde k průchodu tělem cyklu. U těchto cyklů může nastat případ, že tělo cyklu neproběhne ani jednou.



2. Cykly s podmínkou na konci. Při průchodu programu se nejprve provede tělo cyklu a posléze se testuje, jestli se tělo bude opakovat, nebo program poběží dál.



3. Cykly s předem daným počtem opakování jsou takové, kdy předem víme, kolikrát se bude cyklus opakovat.
4. Cykly s neznámým počtem opakování jsou takové, u nichž předem nelze odhadnout počet opakování. Příkladem jsou zadávací cykly, kdy uživatel nezná počet zadávaných hodnot, ví jen to, že zadávání ukončí zadáním speciálního znaku, hodnoty nebo slova.

Cyklus For

Jedná se o jeden z nejčastěji používaných cyklů. Je to cyklus s podmínkou na začátku a s předem daným počtem opakování. Definice cyklu se skládá ze tří částí oddělených středníkem.

1. Inicializační část. V této části je možné založit proměnnou a nastavit její počáteční hodnotu.
2. Podmínka. V této části cyklu je vyhodnocována podmínka. Pokud podmínka platí, cyklus se bude vykonávat.
3. Inkrementační/dekrementační část slouží k změně hodnoty proměnné vytvořené v inicializační části

```
// v inicializační části je založena proměnná i a uložena do ní hodnota 0
// je testována podmínka, jestliže je i menší než 10, tak se cyklus vykonává
// v inkrementační části je proměnná i zvětšena při každém průchodu o 1
// cyklus projde tělo 10x, proměnná i se bude postupně zvětšovat od 0-9
// pokud bude i rovno 10, přestane platit podmínka a cyklus se ukončí
```

```
for (int i = 0; i < 10; i++)
{
    //zde se nachází tělo cyklu
}
```

Program „Součet sudých čísel“

Na to, abychom si procvičili příkaz for, vytvoříme program, který bude počítat součet sudých čísel v zadaném intervalu. Nejprve je třeba si říct, jak zjistíme, je-li číslo sudé. Stačí vytvořit podmínku, kdy budeme testovat, zdali celočíselný zbytek po dělení 2 bude roven 0 (předchozí příklad na procvičení) Zvolený interval jednoduše zadáme do cyklu for.

1. Založte si nový projekt s názvem SoucetSudCisel.
2. Opíšeme následující program.

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Zadejte min intervalu");
        // založím proměnnou min a přetypuji uživatelský vstup na integer
        int min = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("Zadejte max intervalu");
        // založím proměnnou max a přetypuji uživatelský vstup na integer
        int max = Convert.ToInt32(Console.ReadLine());

        // založíme proměnnou součet a uložíme do ní hodnotu 0
        int soucet = 0;
```

```

// vytvoříme cyklus, kde se proměnná i postupně mění od min do max
for (int i = min; i <= max; i++)
{
    // tento if zjišťuje, jestli se jedná o sudé číslo
    if (i % 2 == 0)
    {
        // zde přičítáme sudé číslo do proměnné součet
        soucet = soucet + i;
    }
}

//vypíšu výsledek
Console.WriteLine("součet sudých čísel v inter. od {0} do {1} je {2}",min, max, soucet);

// počkám na stisk libovolné klávesy pro ukončení programu
Console.ReadKey();
}
}

```

Cyklus While

While je cyklus s podmínkou na začátku, ale na rozdíl od for není předem daný počet opakování. Tento cyklus nemá takzvaný čítač a počet opakování se určuje na základě platnosti podmínky cyklu.

```

//cyklus se vykonává, dokud je splněná podmínka
while (podminka)
{
    // zde je tělo cyklu
}

```

Pokud budeme chtít vykonat 10x průchod tělem cyklu, bude to vypadat takto.

```

int A = 0;

// cyklus se vykonává, pokud je splněná podmínka
while (A < 10)
{
    // tady zvětšíme A o jedničku
    A++;
}

```

Zkusíme upravit předchozí příklad, jak by vypadal s použitím cyklu while.

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Zadejte min intervalu");
        // založím proměnnou min a přetypuji uživatelský vstup na integer
        int min = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("Zadejte max intervalu");
        // založím proměnnou max a přetypuji uživatelský vstup na integer
        int max = Convert.ToInt32(Console.ReadLine());

        // založíme proměnnou součet a uložíme do ní hodnotu 0
        int soucet = 0;

        // vytvoříme cyklus s podmínkou
        while (min <= max)
        {
            // tento if zjišťuje, jestli se jedná o sudé číslo
            if (min % 2 == 0)
            {
                // zde přičítáme sudé číslo do proměnné součet
                soucet = soucet + min;
            }

            // při každém průchodu zvětšíme min o jedna
            min++;
        }

        //vypíšu výsledek
        Console.WriteLine("součet sudých čísel v inter. od {0} do {1} je {2}",min, max, soucet);

        // počkám na stisk libovolné klávesy pro ukončení programu
        Console.ReadKey();
    }
}
```

V případě, že jste vše správně napsali, bude program pracovat stejně i s použitím nového cyklu while.

Cyklus Do-While

Do-While je druh cyklu s podmínkou na konci. Jak jsme si řekli v předcházejícím textu, tento druh cyklu provede min jednou průchod tělem cyklu a pak teprve dochází k vyhodnocení podmínky. Tento cyklus se nepoužívá tak často jako předchozí cykly. Zápis cyklu vypadá následovně.

```
// cyklus se vykonává, dokud je splněná podmínka
do
{
    } while (podmínka);
```

Pokud budeme chtít vykonat 10x průchod tělem cyklu, bude to vypadat takto:

```
int A = 0;

// cyklus se vykonává, pokud je splněná podmínka
do
{
    // tady zvětšíme A o jedničku
    A++;
} while (A < 10);
```

Všimněte si několika zajímavých věcí.

1. Za cyklem se píše středník (středník slouží k ukončení příkazu)
2. Proměnná A je zvětšena o jedničku před podmínkou.

Máme za sebou základní cykly, zbývá nám už jen jeden, ale ten si necháme na později.

Už jste někdy programovali hru? V následující kapitole se do toho pustíme.

Hra „Hádání čísla“

Smyslem hry bude uhádnout na daný počet pokusů náhodně vygenerované číslo od 1 do 100.

Generování náhodného čísla

Nejprve si musíme říct, jak náhodně vygenerovat číslo. Generování něčeho náhodného je pro počítač celkem náročný úkol. Existuje mnoho algoritmů, které se snaží problém náhody řešit, ale jejich matematická obtížnost je velká. Naštěstí máme k dispozici třídu `Random`, která se o to postará.

```
// vytvoříme objekt nahoda ze třídy Random
Random Nahoda = new Random();

// do int nahodne_cislo uložíme náhodné číslo vygenerované metodou Next
// v rozsahu 1-100
int nahodne_cislo = Nahoda.Next(1, 101);
```

Je to jednoduché a můžeme se pustit do programování naší první hry.

1. Založte si nový projekt s názvem `HadaniCisla`.
2. Opíšeme následující program.

```
class Program
{
    static void Main(string[] args)
    {
        bool dobre = false;
        Console.WriteLine("Zadej počet pokusů");
        int pokus = Convert.ToInt32(Console.ReadLine());

        // vytvoříme objekt nahoda ze třídy Random
        Random Nahoda = new Random();

        // do int nahodne_cislo uložíme náhodné číslo vygenerované metodou Next
        // v rozsahu 1-100
        int cislo = Nahoda.Next(1, 101);

        // cyklus počtu pokusů
        for (int i = 1; i <= pokus; i++)
        {
            // zadání hádaného čísla
            Console.WriteLine("Tak co myslíš, jaké číslo to bude?");
            int cisloz = Convert.ToInt32(Console.ReadLine());

            // test výhry
            if (cisloz == cislo)
            {
                Console.WriteLine("Ses machr! uhadnul jsi číslo {0} na {1} pokusů", cislo, i);
                dobre = true;
                break; // ukončí předčasně cyklus
            }
        }
    }
}
```

```

        // test menší nebo větší
        if (cisloz > cislo)
        {
            Console.WriteLine("Je to menší číslo");
        }
        else
        {
            Console.WriteLine("Je to větší číslo");
        }
    }
    // test jestli došly pokusy
    if (!dobre)
    {
        Console.WriteLine("Pokusy nestačily, konec hry");
    }

    // počkám na stisk libovolné klávesy pro ukončení programu
    Console.ReadKey();
}
}

```

Příklady na procvičení

1. Napište program, který sečte všechna čísla v rozsahu [1 – zadaná hodnota].

Nápověda: Vytvořte před cyklem proměnou soucet = 0, a v cyklu přičítejte do této proměnné i cyklu. soucet = soucet + i

2. Napište program, který zjistí kolik čísel v zadaném rozsahu je dělitelné 3.

Nápověda: Číslo je dělitelné 3 pokud platí – $\rightarrow \text{cislo} \% 3 == 0$

3. Napište program, který zjistí min a max z deseti zadávaných čísel.

Nápověda: podívejte se na program „Největší číslo ze tří“ co se v kódu opakuje.

Jednorozměrné pole

Jednorozměrné pole je datová struktura pro uchovávání většího množství dat stejného datového typu. Je to jednoduché: představte si, že v paměti počítače máte za sebou uloženy proměnné stejného datového typu. Jednotlivé proměnné nemají přiřazené žádné jméno, jen celá skupina se nějak jmenuje. Pokud se celá skupina proměnných jmenuje jedním jménem, nastává problém, jak pracovat s konkrétní proměnnou. Není to nic těžkého, stejně jako u stringů nám pomohou indexy.

Založení pole

Jednorozměrné pole si založím třeba takto:

```
// vytvoří jednorozměrné pole typu integer s názvem pole1
// velikost pole je 6 prvků, uloží do pole1 hodnoty 8,7,6,2,1,9
int[] pole1 = { 8, 7, 6, 2, 1, 9 };
```

V paměti si ho můžete představit takto.

Index pole []	0	1	2	3	4	5
Hodnota pole	8	7	6	2	1	9

V případě, že budete potřebovat vytvořit prázdné pole o určitém rozměru, použijte následující zápis.

```
// pole má velikost 10 prvků datového typu integer a je vyplněno nulami
int[] pole1 = new int[10];
```

Velikost pole

Pole jsou indexována od nuly. V případě, že budeme potřebovat zjistit celkovou velikost pole, stačí použít vlastnost Length.

```
int velikost = pole1.Length;
```

V našem případě bude v proměnné velikost hodnota 6.

Čtení z pole

Můžeme přečíst libovolnou položku pole.

```
int polozka = pole1[1];
```

V proměnné polozka bude hodnota 7.

Zápis do pole

Můžeme přepsat libovolnou položku pole.

```
pole1[4] = 16;
```

Pole bude potom vypadat takto.

Index pole []	0	1	2	3	4	5
Hodnota pole	8	7	6	2	16	9

Setřídění pole

Pole lze jednoduše vzestupně setřídít. Použijeme na to třídu Array a metodu Sort().

```
Array.Sort(pole1);
```

Pole po setřídění vypadá takto.

Index pole []	0	1	2	3	4	5
Hodnota pole	2	6	7	8	9	16

Otočení pole

Ještě můžeme pole reverzovat (otočit). Použijeme na to stejnou třídu Array a metodu Reverse().

```
Array.Reverse(pole1);
```

Pole po otočení vypadá takto.

Index pole []	0	1	2	3	4	5
Hodnota pole	16	9	8	7	6	2

Výpis pole

Pro výpis pole použijeme cyklus. Budeme procházet jednotlivá políčka, dokud nedojdeme na konec pole. Nejprve použijeme cyklus for a později si ukážeme i nový cyklus foreach.

```
class Program
{
    static void Main(string[] args)
    {
        // vytvoří jednorozměrné pole typu integer s názvem pole1
        // velikost pole je 6 prvků, uloží do pole1 hodnoty 8,7,6,2,1,9
        int[] pole1 = { 8, 7, 6, 2, 1, 9 };

        // cyklus prochází postupně celé pole
        for (int i = 0; i < pole1.Length; i++)
        {
            Console.Write("{0},", pole1[i]);
        }

        // počkám na stisk libovolné klávesy pro ukončení programu
        Console.ReadKey();
    }
}
```

Cyklus Foreach

Cyklus foreach se používá na výpis datových kolekcí. Mezi datové kolekce patří i jednorozměrné pole. Jeho použití je velice snadné a nebudeme potřebovat pracovat s indexy. Foreach prochází postupně celé pole a sám si hlídá počet průchodů podle jeho velikosti. Použití si ukážeme na předchozím příkladu výpisu pole.

```
class Program
{
    static void Main(string[] args)
    {
        // vytvoří jednorozměrné pole typu integer s názvem pole1
        // velikost pole je 6 prvků, uloží do pole1 hodnoty 8,7,6,2,1,9
        int[] pole1 = { 8, 7, 6, 2, 1, 9 };

        // foreach postupně projde všechny položky
        foreach (int polozka in pole1)
        {
            Console.Write("{0},", polozka);
        }

        // počkám na stisk libovolné klávesy pro ukončení programu
        Console.ReadKey();
    }
}
```

Program „Součet pole“

V dalším programu si ukážeme, jak sečíst všechna čísla v poli. Není na tom nic složitého stačí pole postupně procházet a jednotlivé hodnoty načítat do proměnné soucet.

```
static void Main(string[] args)
{
    // vytvoří jednorozměrné pole typu integer s názvem pole1
    // velikost pole je 6 prvků, uloží do pole1 hodnoty 8,7,6,2,1,9
    int[] pole1 = { 8, 7, 6, 2, 1, 9 };

    // založení proměnné součet
    int soucet = 0;

    // foreach postupně projde všechny položky
    foreach (int polozka in pole1)
    {
        soucet = soucet + polozka;
    }

    // vypsání součtu
    Console.WriteLine("Součet pole je {0}", soucet);

    // počkám na stisk libovolné klávesy pro ukončení programu
    Console.ReadKey();
}
```

Program „Min a Max v poli “

Často v programování budete potřebovat z nějakých dat zjistit jejich minimální, nebo maximální hodnotu. Ukážeme si dvě možnosti, jak v poli toto vyřešit.

První možností je pomocí cyklu.

```
static void Main(string[] args)
{
    // vytvoří jednorozměrné pole typu integer s názvem pole1
    // velikost pole je 6 prvků, uloží do pole1 hodnoty 8,7,6,2,1,9
    int[] pole1 = { 8, 7, 6, 2, 1, 9 };

    // založení proměnné min
    int min = pole1[0];

    // založení proměnné max
    int max = pole1[0];

    // foreach postupně projde všechny položky
    foreach (int polozka in pole1)
    {
        if (polozka > max) max = polozka;
        if (polozka < min) min = polozka;
    }

    // vypsání min a max v poli
    Console.WriteLine("Min v poli je {0} a max je {1}", min,max);

    // počkám na stisk libovolné klávesy pro ukončení programu
    Console.ReadKey();
}
```

Druhou možností je pomocí setřídění.

```
static void Main(string[] args)
{
    // vytvoří jednorozměrné pole typu integer s názvem pole1
    // velikost pole je 6 prvků, uloží do pole1 hodnoty 8,7,6,2,1,9
    int[] pole1 = { 8, 7, 6, 2, 1, 9 };

    //setřídím pole od min do max
    Array.Sort(pole1);

    // založení proměnné min, první bude min
    int min = pole1[0];

    // založení proměnné max, poslední bude max
    int max = pole1[pole1.Length - 1];

    // vypsání min a max v poli
    Console.WriteLine("Min v poli je {0} a max je {1}", min, max);

    // počkám na stisk libovolné klávesy pro ukončení programu
    Console.ReadKey();
}
```

Program „Naplnění pole náhodnými čísly“

Pokud budete potřebovat naplnit pole náhodnými čísly v nějakém rozsahu, tak použijete třídu Random. Pomocí její metody Next budete generovat náhodná čísla a pomocí cyklu je ukládat do pole. Vše si můžete prohlédnout v následujícím programu.

```
static void Main(string[] args)
{
    // vytvoříme jednorozměrné pole typu integer o velikosti 10 prvků
    int[] pole = new int[10];

    // vytvořím objekt Rnd
    Random Rnd = new Random();

    for(int i=0; i < pole.Length; i++)
    {
        // při každém průchodu generuji náhodné číslo z rozsahu 1-9
        // a ukládám do pole
        pole[i] = Rnd.Next(1, 10);

        // výpis pole
        Console.Write("{0},", pole[i]);
    }

    // počkám na stisk libovolné klávesy pro ukončení programu
    Console.ReadKey();
}
```

Program „Překladač do Morseovy abecedy“

V Morseově abecedě je každý znak interpretován jako skupina symbolů složená z čárek a teček (krátký a dlouhý signál). Tuto abecedu navrhl už na začátku devatenáctého století americký malíř a vynálezce Samuel Morse. Dodnes se pro svoji jednoduchost používá především v nouzových stavech. Náš program bude pro zjednodušení kódovat pouze znaky A-Z.

A	.-	H		O	---	V	...-
B	-...	I	..	P	.-.	W	.-.
C	-. -.	J	.-.-	Q	--.	X	-. -.
D	-..	K	-.-	R	.-.	Y	-. -.-
E	.	L	.-..	S	...	Z	---.
F	..-.	M	--	T	-		
G	--.	N	-.	U	..-		

Tabulka Morseovy abecedy

```

static void Main(string[] args)
{
    // pole Morseovy abecedy A "-." index 0, B "-..." index 1 .....
    string[] pole = {
        "-.", "-...", "-.-.", "-..", ".-", ".-.-.", "-.-.",
        "....", ".-", ".-.-.", "-.-.", "-.-.", "-.-.", "-.-.",
        "-.-.", "-.-.", "-.-.", "-.-.", "-.-.", "-.-.", "-.-.",
        "-.-.", "-.-.", "-.-.", "-.-.", "-.-.", "-.-." };

    // do proměnné zadano se uloží vstupní text a převede se celý na velká
    // písmena metodou ToUpper
    string zadano = Console.ReadLine().ToUpper();

    // založíme pomocnou proměnnou
    string preklad = string.Empty;

    // cyklus prochází vstupní text znak po znaku až do konce textu
    for (int i = 0; i < zadano.Length; i++)
    {
        // pokud je v textu znak mezery
        if (zadano[i] == ' ')
        {
            // do proměnné preklad se přidá znak lomítka
            // tento řádek je zkrácený zápis tohoto
            // preklad = preklad + "/";
            preklad += "/";
        }
        // pokud není znak mezery
        else
        {
            // testuje se, jestli se jedná o znak v rozmezí A-Z
            if (zadano[i] >= 'A' && zadano[i] <= 'Z')
            {
                // do proměnné morse se uloží znaky Morseovy abecedy z pole
                // index se vypočítá tak, že od aktuálního znaku odečtu 'A'
                // pokud se podíváte do ASCII tabulky, tak A = 65
                // například pokud je aktuální znak C (67 ASCII) 67-65 = 2
                // na indexu 2 v „poli“ je v Morseově abecedě "-.-." = C
                string morse = pole[zadano[i] - 'A'];

                // do proměnné preklad přidám řetězec morseovky pro znak
                preklad += morse;
            }
            // pokud to není znak A-Z, tak ho neumím přeložit a přidám "?"
            else
            {
                preklad += "?";
            }
        }
    }

    // vypíšu překlad
    Console.WriteLine(preklad);

    // počkám na stisk libovolné klávesy pro ukončení programu
    Console.ReadKey();
}

```


Dvourozměrné pole

Stejně jako jednorozměrné pole je toto pole datová struktura pro uchování dat stejného datového typu. Přidáním dalšího rozměru je možné si dvourozměrné pole představit jako tabulku dat. K jednotlivým hodnotám v poli se dostaneme pomocí dvou indexů. První index zpravidla určuje sloupec a druhý řádek tabulky. Pro představu si můžeme ukázat pole o velikosti 5x5.

Index	0	1	2	3	4
0	2	4	7	4	6
1	0	1	3	2	7
2	6	0	8	5	9
3	8	9	1	8	0
4	4	2	9	5	2

Příklad pole velikost 5x5

Založení dvourozměrného pole

Dvourozměrné pole založíme podobně jako jednorozměrné. Pole datového typu integer s již uloženými hodnotami je možné vytvořit následovně.

```
// toto je předvyplněné pole 5x5 datového typu integer
int[,] pole2d = { {2,4,7,4,6},
                  {0,1,3,2,7},
                  {6,0,8,5,9},
                  {8,9,1,8,0},
                  {4,2,9,5,2} };
```

Index	0	1	2	3	4
0	2	4	7	4	6
1	0	1	3	2	7
2	6	0	8	5	9
3	8	9	1	8	0
4	4	2	9	5	2

Pokud budeme potřebovat prázdné pole určitého rozměru, stačí to napsat takto. Pole bude naplněno nulami.

```
// toto je prázdné pole 5x5 datového typu integer
int[,] pole2d2 = new int[5, 5];
```

Index	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

Zjištění velikosti 2D pole

Pro zjištění celkové velikosti je možné použít jako u jednorozměrného pole vlastnost Length.

```
int velikost = pole2d.Length;
```

V našem případě vrátí hodnotu 25.

Častěji budeme potřebovat informaci o počtu sloupců a řádků. Pro zjištění této informace je dobré použít metodu GetLength(parametr) se vstupním parametrem 0 pro řádky a 1 pro sloupce.

```
// počet řádků  
int pocet_radku = pole2d.GetLength(0);  
  
// počet sloupců  
int pocet_sloupcu = pole2d.GetLength(1);
```

Čtení z 2D pole

Můžeme přečíst libovolnou položku pole.

Index	0	1	2	3	4
0	2	4	7	4	6
1	0	1	3	2	7
2	6	0	8	5	9
3	8	9	1	8	0
4	4	2	9	5	2

```
int polozka = pole2d[3,2];
```

V proměnné polozka bude hodnota 1.

Zápis do 2D pole

Můžeme přepsat libovolnou položku pole.

```
Pole2d[1,3] = 10;
```

Pole bude vypadat takto

Index	0	1	2	3	4
0	2	4	7	4	6
1	0	1	3	10	7
2	6	0	8	5	9
3	8	9	1	8	0
4	4	2	9	5	2

Výpis 2D pole

Pokud budeme potřebovat zobrazit pole jako tabulku, použijeme dva cykly v sobě. První cyklus bude procházet jednotlivé řádky a vnitřní druhý cyklus vždy projde sloupce vybraného řádku. Danou metodu si vyzkoušíme v následujícím příkladu.

```
static void Main(string[] args)
{
    // toto je předvyplněné pole 5x5 datového typu integer
    int[,] pole2d = { {2,4,7,4,6},
                      {0,1,3,2,7},
                      {6,0,8,5,9},
                      {8,9,1,8,0},
                      {4,2,9,5,2} };

    // tento cyklus prochází řádky
    for (int i = 0; i < pole2d.GetLength(0); i++)
    {
        // v každém řádku projde tento cyklus všechny sloupce
        for (int j = 0; j < pole2d.GetLength(1); j++)
        {
            // postupně vypisuje na řádek čísla
            Console.Write("{0},", pole2d[i, j]);
        }
        // po každém řádku musí odřádkovat
        Console.WriteLine();
    }

    // počkám na stisk libovolné klávesy pro ukončení programu
    Console.ReadKey();
}
```

Příklady na procvičení

1. Vytvořte program, který vypíše součty jednotlivých řádků 2D pole.
2. Vytvořte 2D pole naplněné tak, aby v první řádek obsahoval pouze číslo 1, druhý 2, třetí 3 ...
3. Naplňte 2D pole přesně daným počtem 1. Zadáme velikost pole a počet jedniček.

Hra Lodě

Určitě znáte hru lodě. Jedná se o hru, kde vaším úkolem je sestřelit určitý počet skrytých lodí. Vyhráváte ve chvíli, kdy jsou všechny lodě potopené. Naše hra je záměrně trochu jednodušší. Nenajdete v ní různé typy lodí, ale jen dvourozměrné pole naplněné náhodně číslem 2 což představuje loď a 0 což představuje vodu. Projděte si následující kód a pak si zkuste sami hru vylepšit.

```
static void Main(string[] args)
{
    // vytvořím si prázdné pole 5x5
    int[,] pole = new int[5, 5];

    Random rnd = new Random();

    Console.WriteLine("zadej počet lodí, max je 20");
    int PocetLodi = Convert.ToInt32(Console.ReadLine());

    Console.WriteLine("zadej počet pokusů");
    int PocetPokusy = Convert.ToInt32(Console.ReadLine());

    // cyklus pro náhodné vygenerování daného počtu lodí
    for (int i = 1; i <= PocetLodi; i++)
    {
        // vygenerování náhodných indexů
        int x = rnd.Next(0, pole.GetLength(0));
        int y = rnd.Next(0, pole.GetLength(1));
        // kontrola jestli na indexu už není loď (2)
        if (pole[x, y] == 2) // pokud je, tak vrať cyklus o jeden krok zpět
        {
            i--;
        }
        else // pokud není tak jí tam zapiš
        {
            pole[x, y] = 2;
        }
    }

    // cyklus pro střelbu
    for (int k = 1; k <= PocetPokusy; k++)
    {
        // vykreslení pole i s loděma pro testování
        for (int i = 0; i < pole.GetLength(0); i++)
        {
            for (int j = 0; j < pole.GetLength(1); j++)
            {
                Console.Write("{0},", pole[i, j]);
            }

            Console.WriteLine();
        }

        // zadání střelby
        Console.WriteLine("zadej radek");
        int x = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("zadej sloupec");
        int y = Convert.ToInt32(Console.ReadLine());
    }
}
```

```

// kontrola indexů střelby
if(x < 0 || x >= pole.GetLength(0) || y < 0 || y >= pole.GetLength(1))
{
    Console.WriteLine("Střelba mimo pole");
    k--;
    continue;
}

// test kam jsem se trefil
switch (pole[x, y])
{
    case 0: Console.WriteLine("Voda"); pole[x, y] = 1; break;
    case 1: Console.WriteLine("Sem už jsi střílel, byla voda"); break;
    case 2: Console.WriteLine("Zásah"); pole[x, y] = 3; PocetLodi--; break;
    default: Console.WriteLine("Sem už jsi střílel, byl zásah"); break;
}

// vyhodnocení výhry
if (PocetLodi == 0)
{
    Console.WriteLine("Vyhrál jsi");
    break;
}
}
Console.WriteLine("Konec hry");

// počkám na stisk libovolné klávesy pro ukončení programu
Console.ReadKey();
}

```

Metody proměnné typu string

Každá proměnná typu string obsahuje spoustu zajímavých metod a vlastností, které se hodí při práci s řetězci. Ukážeme si nyní využití těch nejvíce používaných.

Porovnání dvou řetězců

Vytvoříme dva řetězce k porovnání.

```
string retez1 = "ahoj";  
string retez2 = "nazdar";
```

Použijeme metodu Compare pro porovnání. Metoda vrací tři hodnoty.

-1 v případě neshody a v případě, že první řetězec začíná v abecedním pořadí dříve.

0 v případě shody obou řetězců

1 v případě neshody a v případě, že první řetězec začíná v abecedním pořadí později.

```
// použijeme metodu Compare pro porovnání  
if (string.Compare(retez1, retez2) == 0) Console.WriteLine("Stejné");  
else Console.WriteLine("Různé");
```

Metoda umí porovnat i část řetězce s jiným, kde 0 je index znaku od kterého budu porovnávat a 4 udává počet znaků které budu porovnávat. V následujícím příkladu mi console vypíše, že oba řetězce jsou stejné, protože porovnávám pouze první 4 znaky.

```
string retez1 = "ahoj";  
string retez2 = "ahoj jak se mas";  
  
// použiji metodu Compare pro porovnání  
if (string.Compare(retez1, 0, retez2, 0, 4) == 0) Console.WriteLine("Stejné");  
else Console.WriteLine("Různé");
```

Vložení nového řetězce do původního

```
// vytvořím si dva řetězce  
string retez1 = "aaaaaa";  
string retez2 = "bbb";  
  
// do řetězce jedna od začátku (index 0) vložím řetězec dva  
retez1 = retez1.Insert(0, retez2);  
Console.WriteLine(retez1);
```

Výsledek uložení v řetězci jedna bude "bbbaaaaaa"

Odstranění části řetězce

```
string retez1 = "ahoj jak se dnes mas";

// z řetězce odstraní část začínající na indexu 12 která má velikost 5 znaků
retez1 = retez1.Remove(12, 5);
```

Výsledek uložený v řetězci jedna bude "ahoj jak se mas"

Vyjmutí části řetězce

```
string retez1 = "ahoj jak se dnes mas";

// z řetězce vyjme část začínající na indexu 0 která má velikost 4 znaky
retez1 = retez1.Substring(0,4);
```

Výsledek uložený v řetězci jedna bude "ahoj"

Nahrazení znaku nebo části řetězce jiným

```
string retez1 = "ahoj jak se dnes mas";

// nahradí slovo ahoj za Nazdar ve všech výskytech v řetězci
retez1 = retez1.Replace("ahoj", "Nazdar");
```

Výsledek uložený v řetězci jedna bude "Nazdar jak se dnes mas"

Rozdělení řetězce na jednorozměrné pole podle určeného znaku

Velmi vhodné použít pro zadání více čísel na jednom řádku

```
// vytvořím si řetězec
string retez1 = "10,20,30";

// pomocí Split rozdělím řetězec do pole podle čárky
// v každém políčku pole budou znaky čísel [10][20][30]
string[] poleStr = retez1.Split(',');

// cyklem projdu celé pole
int soucet = 0;

foreach(string pol in poleStr)
{
    // do proměnné součet přičtu jednotlivá čísla
    soucet = soucet + Convert.ToInt32(pol);
}

Console.WriteLine(soucet);
```

Závěr

Pokud jste dočetli až sem a vyzkoušeli všechny programy, tak nezbývá než vám pogratulovat. Programování je dovednost, která se nezapomíná a pokud vás bude bavit, tak se postupně budete zlepšovat. V dnešní době máte na výběr v podstatě jen mezi dvěma volbami. Buďto budete uživatelé softwaru, nebo budete ti, co software programují. Na každém z vás je, jakou volbu si zvolíte. Vězte, že programování je zábava, a využijte možností se naučit něco nového. Pokud jste studenti, máte v podstatě všechny vývojové nástroje od firmy Microsoft zdarma. To, co jste právě přečetli, je jen začátek.